SUMMARY: OPERATORS

SESSION OVERVIEW:

In this session, the students will be able to:

- Understand what are operators and why they are used
- Understand different types of operators.
- Understand order of Operations Followed in Python.
- Constraints in programming language.

KEY POINTS AND EXAMPLES:

Understand what are operators and why they are used:

Operators are tools in programming that allow you to perform actions like adding numbers, comparing values, or assigning values to variables.
Operators are utilized for executing operations on both variables and values.

In the example below, we use the + operator to add together two values:

```
print(20 + 5)
```

The output will be:

```
25
```

Understand different types of operators:

There are 7 types of operators:

1) Arithmetic Operators
2) Comparison Operators
3) Logical Operators
4) Bitwise Operators
5) Assignment Operators
6) Membership Operators
7) Identity Operators

Arithmetic operators: These operators are used for mathematical calculations such as addition, subtraction, multiplication, division, modulus (remainder), and exponentiation.

Operator	Description
+ (Addition)	This operator combines two numbers together to give you a total. It's like putting things together to find out how many you have in total.
- (Subtraction)	This operator takes one number away from another. It helps you find the difference between two quantities.
* (Multiplication)	This operator performs the operation of multiplying two operands together to produce a result.
/ (Division)	This operator performs the operation of dividing the numerator by the denominator.resulting in a quotient that may include decimal places, also known as floating-point division.
% (Modulus)	This operator gives you the remainder after dividing one number by another.
** (Exponentiation)	This operator raises a number to a power. It's like multiplying a number by itself a certain number of times.
// (Floor division)	This operator divides one number by another and rounds down to the nearest whole number. It's like regular division, but it always gives you a whole number result, discarding any remainder.

How these operators work:-

```
#Define two numbers
x = 10
y =  3
# Perform arithmetic operations
print(x + y)  # Addition
print(x - y)  # Subtraction
print(x * y)  # Multiplication
print(x / y)  # Division
print(x % y)  # Modulus (remainder)
print(x ** y) # Exponentiation
```

```
print(x//y) # Floor division
```

The output will be:

```
13
7
30
3.3333333333333335
1
1000
3
```

```
print(5/5)
print(10/2)
print(-10/2)
print(20.0/2)
```

The output will be:

```
1.0
5.0
-5.0
10.0
```

Note: The division operator (/) always produces a floating-point number as the result, no matter if two numbers are integers.

```
a = 10 + "20"
print(a)
```

The output will be:

```
TypeError: unsupported operand type(s) for +: 'int' and 'str'
```

Note: Mixing incompatible types, such as integers (int) and strings (str), can lead to errors

Note: The subtraction operation (-) is not defined for strings. Attempting to subtract one string from another will result in a TypeError because string subtraction is not supported.

Comparison Operators: Comparison operators are used to compare two values or expressions. They return a Boolean value (True or False) based on the comparison result.

Operator	Description
== (Equal to)	It checks if two values are equal.
!= (Not equal to)	It checks if two values are not equal.
> (Greater than)	It checks if one value is greater than another.
< (Less than)	It checks if one value is less than another.
>= (Greater than equal to)	It checks if one value is greater than or equal to another.
<= (Less than or equal to)	It checks if one value is less than or equal to another.

How these operators work:-

```
#Define two numbers
x = 5
y = 10
#Perform comparison operations
print(x == y) #Equal to (==)
print(x != y) #Not equal to (!=)
print(x > y)  # Greater than (>)
print(x < y)  # Less than (<)
print(x >= y) # Greater than or equal to (>=)
print(x <= y) # Less than or equal to (<=)
```

The output will be:

```
False
True
False
True
False
True
```

Logical Operators: Logical operators are used to combine multiple conditional statements or expressions. They allow you to make decisions based on the outcome of multiple conditions.

Operator	Description
and	It returns True if both operands are True.
or	It returns True if at least one operand is True.
not	It returns the opposite Boolean value of the operand. If the operand is True, it returns False, and vice versa.

How these operators work:-

```
#Define three numbers
x = 5
y = 10
z = 15
#Perform comparison operations
print(x < y and y < z)  # and
print(x < y or x > z)   # or
print(not(x < y))       # not
```

The output will be:

```
True
True
False
```

The and operator evaluates to True because both conditions x < y and y < z are True.
The or operator evaluates to True because at least one condition x < y is True.
The not operator negates the result of the condition x < y, which is True, resulting in False.

Bitwise Operators: Bitwise operators are operators used to perform operations on individual binary digits of numbers.

Operator	Description
& (Bitwise AND)	It sets each bit to 1 if both corresponding

	bits are 1.
| (Bitwise OR)	It sets each bit to 1 if at least one corresponding bit is 1.
^ (Bitwise OR)	It sets each bit to 1 if only one corresponding bit is 1 (exclusive XOR).
~ (Bitwise NOT)	It flips the bits of the operand, changing 1s to 0s and 0s to 1s.
<< (Left shift)	It shifts the bits of the operand to the left by a specified number of positions, filling the vacated positions with zeros.
>> (Right shift)	It shifts the bits of the operand to the right by a specified number of positions. The vacant positions are filled differently depending on the implementation: with zeros (logical right shift) or with the sign bit (arithmetic right shift).

How these operators work:

```
# Define two numbers
x = 10  # Binary: 1010
y = 6   # Binary: 0110
#perform bitwise operators
print(x & y) #AND
print(x ^ y)  #XOR
print(~x)  #NOT
print(x << 2) #Left shift
print(x >> 2) #Right shift
```

The output will be:

```
2  #0010
14 #1110
12 #1100
-11 #11111111111111111111111111110101
40 # 101000
2  # 10
```

Note: While bitwise operators are briefly covered in this session, they are not extensively

relevant. We'll provide a simple example of bitwise operators, but we won't delve deeply into their mechanics.

Assignment operators: Assignment operators are used to assign values to variables.

Operator	Example	Same As
= (Assignment Operator)	x=5	x=5
+= (Addition Assignment)	x+=2	x=x+2
-= (Subtraction Assignment)	x-=2	x=x-2
*= (Multiplication Assignment)	x*=2	x=x*2
/= (Division Assignment)	x/=2	x=x/2
%= (Modulus Assignment)	x%=2	x=x%2
//= (Floor Division Assignment)	x//=2	x=x//2
**= (Exponentiation Assignment)	x**=2	x=x**2

How these operators work:

```
# Initialize variables
x = 20
y = 3
#perform assignment operators
x+=y  # Equivalent to x = x + y
print(x)
x -= y # Equivalent to x = x - y
print(x)
x *= y # Equivalent to x = x * y
print(x)
x /= y # Equivalent to x = x / y
print(x)
x %= y # Equivalent to x = x % y
print(x)
x **= y # Equivalent to x = x ** y
```

```
print(x)
x //= y # Equivalent to x = x // y
print(x)
```

The output will be:

```
23
20
60
20.0
2.0
8.0
2.0
```

Membership Operators: These operators are used to test whether a value or variable is found within a sequence such as strings, lists, tuples, sets, or dictionaries. Python provides two membership operators: in and not in.

Operator	Description
in	It checks if a value exists in a sequence. Returns True if the value is found, False otherwise.
not in	It checks if a value does not exist in a sequence. Returns True if the value is not found, False otherwise.

Syntax:

```
# Check if value exists in a sequence
value in sequence

# Check if value does not exist in a sequence
value not in sequence
```

How these operators work:

```
print('a' in 'banana')
print('m' not in 'banana')
```

The output will be:

```
True
True
```

Identity operators: Identity operators are used to compare the memory locations of two objects to check if they are the same object or not. In Python, there are two identity operators: is and is not

Operator	Description
is	It returns True if both operands point to the same object, False otherwise.
Is not	It returns True if both variables are not the same object

Syntax:

```
# Check if two objects are the same
object1 is object2

# Check if two objects are not the same
object1 is not object2
```

How these operators work:

```
# Initialize variables
x = 10
y = 10
# Perforom membership operators
print(x is y)
print(x is not y)
```

The output will be:

```
True
False
```

we can check the memory addresses of objects using the id() function, which returns the

memory address of an object.

```
print(id(x))
print(id(y))
```

Understand order of Operations Followed in Python:

In Python, the order of operations follows the standard mathematical convention, often represented by the acronyms PEMDAS or BODMAS. These acronyms stand for:

1. **Parentheses/Brackets:** Operations within parentheses or brackets are performed first. Any expression within parentheses takes precedence over the rest of the expression. For example, (2 + 3) * 4 will first add 2 and 3, and then multiply the result by 4.

2. **Exponents/Orders:** Exponentiation operations (raising a number to a power) are performed next. Exponents take precedence over multiplication, division, addition, and subtraction. For instance, 2 ** 3 represents 2 raised to the power of 3, resulting in 8.

3. **Multiplication/Division:** Multiplication and division operations are evaluated next. These operations are performed from left to right. For example, 6 / 2 * 3 results in 9, not 1.5, because division is evaluated before multiplication.

4. **Addition/Subtraction:** Addition and subtraction operations are evaluated last among the basic arithmetic operations. Similar to multiplication and division, these operations are performed from left to right. For instance, 5 - 3 + 2 results in 4, not 0, because subtraction is evaluated before addition.

For example,

```
a = ((2 + 3) ** 2 + 4 * 3) / (5 - 2) + 10 // 3
print(a)
```

The output will be:

```
15.333333333333334
```

In above example, First, we begin with the operations within the first set of parentheses, or brackets, which take precedence over other operations. In our expression, (2 + 3) evaluates to

5. Next, we move to the exponentiation operation within the second set of parentheses, where (5 ** 2) results in 25. Continuing with the multiplication operation, 4 * 3 equals 12. Then, we perform addition, adding 25 and 12 to get 37. Subsequently, we evaluate the expression within another set of parentheses, 5 - 2, which equals 3. Dividing 37 by 3, we obtain approximately 12.33. Moving on, we perform floor division, where 10 // 3 equals 3. Finally, we add the results of division and floor division, resulting in approximately 15.33.

Python also considers a few other operations in its order:

1. **Unary plus and minus:** Operations like -x or +x are evaluated right after parentheses but before exponents.

```
x = 5
result = -((-x) ** 2) + -(+x)
print(result)  # Output: -30
```

In this example, In this expression, Python first tackles what's inside the inner parentheses. It sees -x and applies the unary minus, turning x (which is 5) into -5. Then, it squares that value, giving us 25. After that, it applies another unary minus to -25, so now it's -(-25), which equals 25. Meanwhile, the unary plus doesn't change x (which is still 5). So, when you put it all together and subtract -5 from 25, you get 20.

2. **Bitwise operations:** These include shifts (<< and >>), bitwise AND (&), OR (^), and OR (|), which come into play after the basic arithmetic operations but before comparisons.

```
x = 10
y = 5
result = (x + 3) << 2 > (y * 2)
print(result) # output: True
```

In this example, Python first adds 3 to the value of x, resulting in 13. Similarly, y is multiplied by 2, yielding 10. Then, the << operator is applied, performing a left shift on 13. This operation shifts the binary representation of 13 two positions to the left, resulting in 52. After the left shift, the comparison 52 > 10 is evaluated, which is True.

3. **Comparisons, Membership, and Identity:** Operations like <, <=, >, >=, ==, !=, in, not in, is, is not are evaluated after all arithmetic and bitwise operations.

```
x = 10
y = 5
```

```
print((x + 3) > (y * 2) and x is not None and x == 10) # output:True
```

In this example, first, we perform arithmetic operations, where x + 3 results in 13, and y * 2 results in 10. Next, we compare these values using the > operator, which evaluates to True since 13 is indeed greater than 10. Moving on to identity operations, we check if x is not None, which evaluates to True since x holds a value. Then, we perform another comparison operation to check if x is equal to 10, which is true. Finally, the and operator combines all these results, resulting in True since all conditions are met.

4. **Logical operators:** Logical operations like 'not', 'and', and 'or' are evaluated at the end, with 'not' having the highest precedence among them, followed by and, and then or.

```
x = True
y = False
result = (not x) or (y and x)
print(result) # output: False
```

In this example, First, the not operator is applied to x, resulting in not True, which evaluates to False. Next, the and operator is applied to y and x, resulting in False and True, which evaluates to False. Finally, the or operator is applied to the result.

```
print(1 or 2 and 3) # output: 1
```

According to the order of operations, and has a higher precedence than or. Therefore, the and operation is evaluated first. The expression 2 and 3 evaluates to 3 because both operands are non-zero, and in a logical and operation, if both operands are True, the result is the second operand. Then, the expression 1 or 3 is evaluated. The or operator returns the first operand if it evaluates to True, otherwise it returns the second operand. Since 1 is truthy, the result of the expression is 1.

```
print(1 or 0 and 3) # output: 1
```

In a logical AND operation, if any operand is false, the result is false. Since 0 is false, the expression 0 and 3 evaluates to 0. Now, we have 1 or 0. In a logical OR operation, if any operand is true, the result is true. Since 1 is true, the expression 1 or 0 evaluates to 1.

The precedence order is described in the table below, starting with the highest precedence at the top:

Operator	Description
()	Parentheses
+x -x ~x	Unary plus, unary minus, and bitwise NOT
**	Exponentiation
* / // %	Multiplication, division, floor division, and modulus
+ -	Addition and subtraction
<< >>	Bitwise left and right shifts
&	Bitwise AND
^	Bitwise XOR
|	Bitwise OR
== != > >= < <= is is not in not in	Comparisons, identity, and membership operators
not	Logical NOT
and	Logical AND
or	Logical OR

Built-in Functions: There are some instances where operators are used internally in Python's built-in functions.

1. **divmod():** The divmod() function takes two numbers as arguments and returns a tuple containing the quotient and the remainder when dividing the first number by the second.

   Syntax: divmod(a, b)
   where, **a:** Dividend
   b: Divisor

```
result = divmod(17, 3)
print(result)

# Output: (5, 2)
```

2. **round():** The round() function rounds a floating-point number to a specified number of decimal places. If no decimal place is provided, it rounds to the nearest integer.

Syntax: round(number, digits)

where, **number:** The number to round.

digits: The number of decimal places to round to (default is 0).

```
result = round(12.5678, 2)
print(result)

# Output: 12.57
```

Constraints in programming language:

Constraints in programming language are limitations or restrictions applied to collection of objects like variables, data structures, data types etc.

- Constraints are useful in solving problems and handling time constraints.
- They are used for optimizations when solving coding problems.

Constraints on Variables:

```
# Add two numbers
num1 = 7
num2 = 3
result = num1 + num2
print(result)

Constraints:
1 <= num1 <= 100
1 < num2 <= 100
```

In the above snippet, constraints are specifying that num1 must have values in the range of 1(inclusive) to 100(inclusive). While num2 must have values in the range of 1(exclusive) to 100(inclusive).

Constraints on Data Structures:

```
# List - Data Structures
lst = [1,2,3,4,5]

Constraints:
# lst[i] is representing each values in lst
-10^5 <= lst[i] <= 10^5
```

In the above snippet, constraints are specifying that list “lst” must have values in the range of -10^5(inclusive) to 10^5(inclusive).